

SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-001: Client-Side Search and Filtering of the Admin Student List

Field	Details
Change Request ID	CR-001
Change Request Title	Client-Side Search and Filtering of the Admin Student List
Student Name	[Student Name]
Student ID	[Student ID]
Implementation Status	Completed
Date	5 October 2026

1. Change Request Summary

1.1 Business Requirement

Administrators previously had to manually scan the complete student table to locate a particular student. The change was required to provide a faster and easier way to locate students using registration number, name, or email address.

1.2 Existing Behaviour

Before implementation:

- The administrator dashboard loaded students using `GET /api/students/`.
- All loaded students were displayed in a table.
- No search or filtering facility was available.
- Administrators had to manually scan the table.
- Existing Edit and Delete operations used `student_id`.

1.3 Requested Change

A client-side search facility was required above the administrator's student table.

The search must:

- Search registration number, first name, last name, full name, and email.
- Support partial and case-insensitive matching.
- Handle unnecessary spaces.
- Filter already-loaded records without making an API request for each keystroke.
- Display the matching record count.

- Provide Clear Search functionality.
- Display a clear no-result message.
- Preserve existing Edit and Delete functionality.

2. Implementation

2.1 Implementation Summary

The CR-001 requirements were implemented in the administrator dashboard.

A search state was added to store the administrator's search input. The existing student list is filtered on the client side using the already-loaded student records. No additional API request is generated when the administrator types in the search field.

The implementation also provides result counting, Clear Search functionality, and a no-result state while preserving the existing Edit and Delete operations.

2.2 Affected Files

File	Change	Purpose
frontend/src/pages/DashboardPage.jsx	Modified	Implemented search state, normalization, filtering, result count, Clear Search, and no-result handling
frontend/src/styles/dashboard.css	Modified / Verified	Provides styling for the search interface and related dashboard elements

Additional files: None / [List any additional files if modified]

2.3 Technical Changes

Search Normalization

A normalization function was implemented to:

- Convert values to lowercase.
- Remove leading and trailing spaces.
- Normalize repeated spaces between words.

This allows searches to remain case-insensitive and tolerant of unnecessary spaces.

Client-Side Filtering

The filtering logic checks the following student fields:

- Registration number

- First name
- Last name
- Full name
- Email address

Partial matching is performed using text matching against the normalized values.

Performance

The search operates on the already-loaded `students` array. Therefore, typing a search term does not generate additional requests to:

```
GET /api/students/
```

The existing API is used only to load the student list.

Existing CRUD Preservation

Edit and Delete operations continue to use:

```
student.student_id
```

Therefore, filtering changes only the displayed records and does not change the identity of a student.

2.4 Implementation Logic

1. Load the complete student list using the existing `GET /api/students/` request.
2. Store the search term in React state.
3. Normalize the search term and student values.
4. Compare the search term against registration number, first name, last name, full name, and email.
5. Display only matching records.
6. Display the number of matching students.
7. Provide Clear Search functionality to restore the complete list.
8. Display a no-result message when no students match the search.
9. Continue using the original `student_id` for Edit and Delete operations.

2.5 Error and Data Handling

The existing student-loading error handling remains unchanged.

The implementation also handles missing or null searchable values by converting them to safe string values before normalization.

When no students match the search term, a clear no-result message and Clear Search option are displayed.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through manual frontend testing.

Testing Types:

- Functional testing
- Positive testing
- Validation testing
- UI testing
- Regression testing
- Error / no-result testing
- CRUD verification

Environment: Development

Browser: [Chrome / Edge / Firefox]

Test Role: Administrator

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Search using registration number	Matching student records are displayed	Correct records displayed	Pass
TC-02	Search using first/last/full name	Appropriate matching students are displayed	Correct matches displayed	Pass
TC-03	Search using email address	Matching student records are displayed	Correct records displayed	Pass
TC-04	Search using different capitalization and unnecessary spaces	Matching results are still returned	Matching results returned	Pass
TC-05	Search using a partial value	Appropriate partial matches are displayed	Partial matches displayed	Pass
TC-06	Search using a value with no match	No-result message and Clear Search option are displayed	No-result message displayed	Pass
TC-07	Click Clear Search	Complete loaded student list is restored	Full list restored	Pass
TC-08	Edit/Delete from filtered results	Correct student is affected using <code>student_id</code>	Correct student affected	Pass
TC-09	Verify existing administrator CRUD	Existing CRUD functionality continues to work	CRUD functions worked correctly	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	9
Passed	9
Failed	0
Pass Rate	100%

Overall Test Result: Pass / Partially Passed / Failed

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Administrators can search by registration number, name, and email.	Functional testing	Met
AC-002	Search is case-insensitive and tolerates unnecessary spaces.	Search normalization test	Met
AC-003	Partial matching returns appropriate records.	Partial search test	Met
AC-004	Result count matches visible records.	Result count verification	Met
AC-005	No-result searches display a clear message and Clear Search option.	No-result test	Met
AC-006	Clearing search restores the complete list.	Clear Search test	Met
AC-007	Edit and Delete affect the correct student.	Filtered CRUD test	Met
AC-008	Existing administrator CRUD and authorization continue to work.	Regression verification	Met

5. Evidence

A maximum of **two relevant screenshots** should be attached.

Evidence 01 – Search Functionality

Related: AC-001, AC-002, AC-003, AC-004

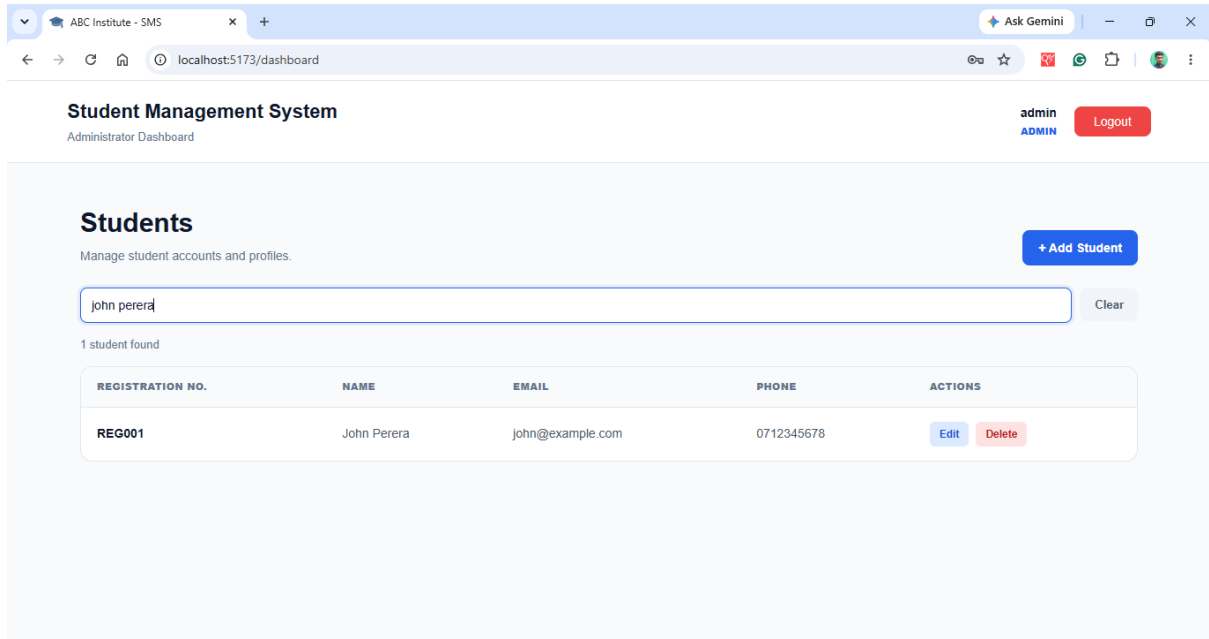


Figure 1: Administrator dashboard showing client-side student search and matching result count.

Evidence 02 – No-Result / Clear Search / CRUD Verification

Related: AC-005, AC-006, AC-007

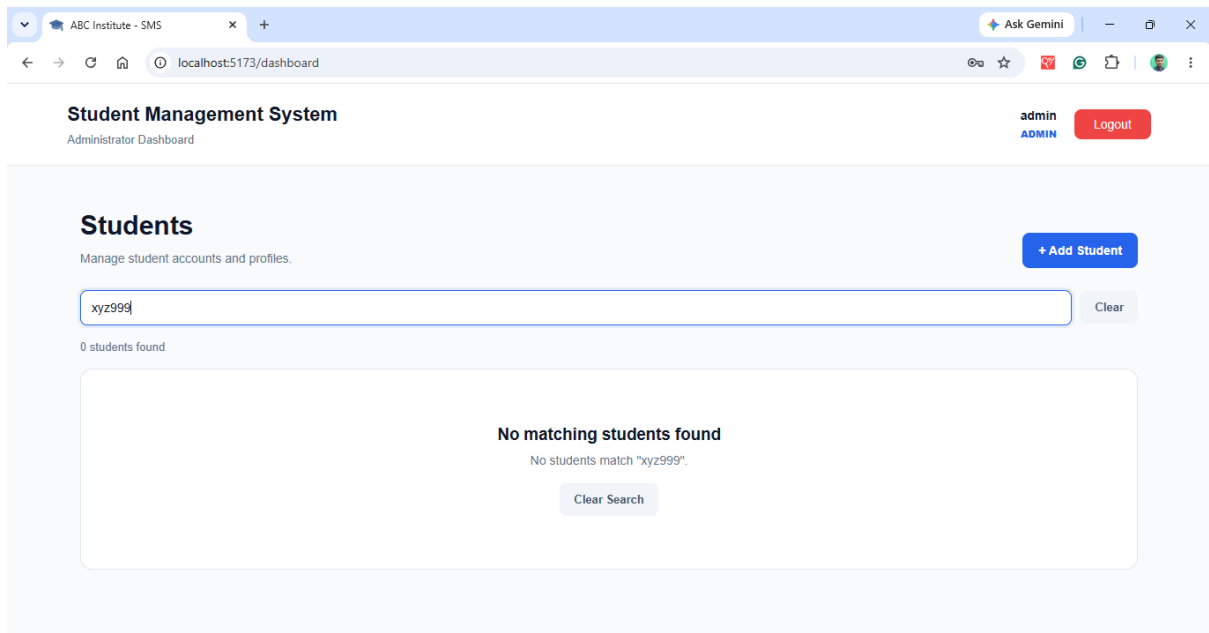


Figure 2: Search no-result state / Clear Search / filtered student CRUD functionality.

6. Final Result

Item	Result
Requirements	Met / Partially Met / Not Met
Implementation	Completed / Partially Completed
Testing	Passed / Partially Passed / Failed
Acceptance Criteria	All Met / Partially Met / Not Met
Existing CRUD	Verified / Not Verified
Final Status	Accepted / Pending / Rejected

6.1 Final Summary

CR-001 was implemented as a client-side search and filtering enhancement for the administrator student dashboard. The implementation searches registration number, first name, last name, full name, and email using case-insensitive and space-normalized matching. Filtering is performed on the already-loaded student records without generating API requests for each search input. Result counting, Clear Search, and no-result handling were added while preserving the existing Edit and Delete functionality using `student_id`. The implemented functionality was manually tested against the defined acceptance criteria.

6.2 Issues / Limitations

[State any remaining issues or limitations.]

If none:

No significant issues identified.

Student Name: _____

Student ID: _____

Date: 5 October 2026

Signature: _____

~ END OF REPORT ~